

第二十四届电子科技大学 ACM-ICPC 程序设计竞赛-初赛题解

A. 染色

显然染色只与子树有关

考虑一个一般节点染色的定义，为了方便我们记 $color(u)$ 表示节点 u 的颜色， S_u 表示 u 的儿子的集合， T_u 表示 u 的子树节点的集合(包括本身)， $w(u)$ 表示 u 的点权，并用 1 代表黑色，0 代表白色。

则：

$$color(u) = \oplus_{x \in S_u} color(x) \oplus w(u) \oplus 1$$

对 $color(x)$ 依次展开最后得到：

$$color(u) = \oplus_{x \in T_u} (w(x) \oplus 1)$$

对于询问 (x, y) ，我们不妨给原无根树随便指定一个根，然后在有根树上分类讨论：

- $x \notin T_y$ ：则该次询问中 y 的子树和当前 y 的子树相同
- $x \in T_y$ ：则该次询问中 y 的子树等同于当前整棵树减去 x 所在的 y 的某个儿子的子树

则我们只需要一个能维护单点修改和子树异或和查询的数据结构，以及能够判断 y 是否是 x 的祖先。

使用 *dfn* 配合任意数据结构加上树上倍增可以很容易做到。

B. 大英帝国

这是一个经典的完全背包问题，物品重量维度为三维。

C. 彩色路径

核心是把“每种颜色出现次数相同”转化为可计数的路径代数条件。

设全树出现过的颜色集合为 S ， $|S| = m$ 。给每种颜色 col 随机赋一个 64 位权值 $w(col)$ ，并令

$$\sum_{col \in S} w(col) = 0$$

如果一条路径覆盖了所有颜色，且每种颜色都出现了相同次数 t ，那么该路径颜色权值和为

$$\sum_{col \in S} t \cdot w(col) = t \cdot \sum_{col \in S} w(col) = 0$$

因此“多彩路径”一定满足路径哈希和为 0。

而且其他路径除非出现哈希冲突的情况，否则不可能路径颜色权值和为 0。

进一步定义树上前缀哈希：

$$pre[u] = pre[fa[u]] + w(c_u)$$

路径哈希可以通过两端点前缀值与 LCA 表示，于是问题变成统计“路径哈希为 0 的点对数”。

统计时用 `small-to-large`（启发式合并）：

1. 每个子树维护一张 `map/hash`（键是某类前缀值，值是出现次数）。
2. 合并小表到大表时，先统计当前小表元素在大表中的可配对数量，再把计数并入。
3. 整体完成后即得到答案。

可以证明，如果哈希的范围是 $2^3 2$ 量级是有不小可能发生哈希冲突的，而在 $2^6 4$ 随机哈希下，冲击几乎变得不可能

复杂度：

- 使用 `std::map` 时一般为 $O(n \log^2 n)$ 。
- 空间复杂度 $O(n)$ 。

D. 计数

本题的解法有很多种包括计算贡献，化简递推式，以及势能计算。本题解将给出化简递推式的做法作为参考。

需要注意的是该做法仅为最朴素的做法。

我们先忽略最后一次正确的猜测，仅考虑这个对所有数字变成 0 所需的错误猜测次数计数。

容易注意到使用不进位加法还是不进位减法不影响答案，为了后文统计的显然我们这里规定使用不进位减法

令 c 为当前该数字中为非 0 的数位的个数，每一轮猜测我们关注 c 的移动，当 c 指向 0 时代表数字变为 0

我们记 dp_x 表示仅考虑小于 k^x 的数字时的答案。我们考虑列出 dp_x 的递推式。

特别补充到 $dp_0 = 0$

为了方便我们从低位到高位依次称其为第 0 到第 $x - 1$ 位，接下来我们分类讨论小于 k^x 的所有数字

我们先考虑最高位是否为 0，若最高位是 0 则该轮中 c 永远不会指向 x ，即我们不关心 $x - 1$ 位的变化，则此时贡献为：

$$dp_{x-1}$$

对于最高位不是 0，我们记 s 为最小的为 0 的数位

不妨先讨论不存在这样的 s ，则所有这样的数字的贡献很显然为每一个数位数字之和：

$$x \frac{k(k-1)}{2} (k-1)^{x-1}$$

存在这样的 s 时，流程会大致如下：

1. $[s+1, x-2]$ 位变为全 0
2. c 连续变化： $s+1 \rightarrow x$
3. c 连续变化： $x \rightarrow 0$

对于第一步，关键是理解小于 s 的数位和 $x - 1$ 数位全部非 0

则我们可以得到此时的贡献：

$$\sum_{s=0}^{x-2} ((k-1)^{s+1} dp_{x-2-s} + (x-1-s)k(k-1)^{s+1}k^{x-2-s} + k^{x-2-s}(s+1) \frac{k(k-1)}{2} (k-1)^s)$$

综上我们得到递推式：

$$dp_x = dp_{x-1} + x \frac{k(k-1)}{2} (k-1)^{x-1} + \sum_{s=0}^{x-2} ((k-1)^{s+1} dp_{x-2-s} + (x-1-s)k(k-1)^{s+1}k^{x-2-s} + k^{x-2-s}(s+1) \frac{k(k-1)}{2} (k-1)^s)$$

等式左右同时除以 $(k-1)^{x-1}$ ：

$$\frac{dp_x}{(k-1)^{x-1}} = \frac{dp_{x-1}}{(k-1)^{x-1}} + \frac{xk(k-1)}{2} + \sum_{s=0}^{x-2} \left(\frac{dp_{x-2-s}}{(k-1)^{x-2-s}} + \frac{k^{x-2-s}}{(k-1)^{x-2-s}} \cdot \frac{(2x-s-1)k}{2} \right)$$

令 $f_x = \frac{dp_x}{(k-1)^x}$ 并改写求和下标：

$$(k-1)f_x = \sum_{i=0}^{x-1} f_i + \sum_{i=0}^{x-2} \frac{k^{i+1}(i+x+1)}{2(k-1)^i} + \frac{xk(k-1)}{2}$$

两边同时除以 $k-1$ ：

$$f_x = \frac{1}{k-1} \sum_{i=0}^{x-1} f_i + \frac{x}{2} \sum_{i=0}^{x-2} \left(\frac{k}{k-1} \right)^{i+1} + \frac{1}{2} \sum_{i=0}^{x-2} \left(\frac{k}{k-1} \right)^{i+1} (i+1) + \frac{xk}{2}$$

分别对等差和等差乘等比求和：

$$\begin{aligned} f_x &= \frac{1}{k-1} \sum_{i=0}^{x-1} f_i + \frac{xk}{2} \left[\left(\frac{k}{k-1} \right)^{x-1} - 1 \right] + \frac{1}{2} \left[k(x-k) \left(\frac{k}{k-1} \right)^{x-1} + k(k-1) \right] + \frac{xk}{2} \\ f_x &= \frac{1}{k-1} \sum_{i=0}^{x-1} f_i + \frac{k}{2} (2x-k) \left(\frac{k}{k-1} \right)^{x-1} + \frac{k(k-1)}{2} \end{aligned}$$

我们令 $A_x = \frac{k}{2} (2x-k) \left(\frac{k}{k-1} \right)^{x-1} + \frac{k(k-1)}{2}$ ，得到：

$$f_x = \frac{1}{k-1} \sum_{i=0}^{x-1} f_i + A_x$$

消除前缀和：

$$\begin{aligned} f_x - f_{x-1} &= \frac{f_{x-1}}{k-1} + A_x - A_{x-1} \\ f_x &= \frac{k}{k-1} f_{x-1} + A_x - A_{x-1} \end{aligned}$$

设 $r = \frac{k}{k-1}$ ，我们求 $B_x = A_x - A_{x-1}$ ：

已知

$$\begin{aligned} A_x &= \frac{k}{2} (2x-k) r^{x-1} + \frac{k(k-1)}{2} \\ A_{x-1} &= \frac{k}{2} (2x-2-k) r^{x-2} + \frac{k(k-1)}{2} \end{aligned}$$

可以得到：

$$A_x - A_{x-1} = \frac{1}{2} (2x+k-2) r^{x-1}$$

$f_0 = 0$ ，令 $f_x = r^x g_x$ 递推转化为：

$$g_x = g_{x-1} + \frac{2x+k-2}{2} \frac{1}{r}$$

累加得到

$$g_x = \sum_{i=1}^x \frac{2i+k-2}{2} \frac{1}{r}$$

利用等差和与等比和公式整理：

$$g_x = \frac{x(x+k-1)}{2r}$$

最终回代整理可以得到最终公式（不包含最终猜测的计数）：

$$dp_x = \frac{(k-1)nk^{n-1}(n-1+k)}{2}$$

则答案为

$$\frac{(k-1)nk^{n-1}(n-1+k)}{2} + k^n$$

E. 全收集

发现合成出的物品代价不低于每个材料的代价，所以用优先队列每次取出当前已知最低代价的物品，将他的代价加入答案，再遍历他参与的合成方案，若所有原料的最低代价都确定则更新合成物品的代价，时间复杂度 $O(n \log n + \sum L)$ 。

F. 恶魔的交易

可以先考虑特殊的情况，如果一开始 Smallfish 不管拿哪堆石子，最后都得到 0 的话，就只能输了。

其次，考虑数组长度是偶数的情况。如果数组中所有数的异或和是 0，那么 Smallfish 会直接胜利。否则他一定可以选择一个数去掉后使剩下数字的异或和不为 0，下面用反证法证明。

设现在所有数字的异或和是 S ，拿走第 i 个数字后剩下的异或和为 S_i ，假设所有 S_i 都是 0，那么有 $S_1 \oplus S_2 \oplus \cdots \oplus S_n = 0$ 。因为数组长度是偶数，所以左边的式子中每个数会出现 $n-1$ 也就是奇数次，异或后得到 $a_1 \oplus a_2 \oplus \cdots \oplus a_n = S = 0$ ，这与上面的前提矛盾，所以可以证明数组长度是偶数时是一定可以找到解的，也就是必胜。

如果数组长度是奇数的话，那就只有一开始全是 0 的情况下 Smallfish 才会赢了，即使他找到拿掉后不为 0 的方案，下一步也会给山羊留下一个偶数长度的数组，上面已经证明了偶数长度是必胜的。

综上，只需要直接求数组异或和并判断数组长度，时间复杂度为 $O(n)$ 。

G. 哈尔滨惨案

题目给出的坐标系满足 $x + y + z = 0$ ，实际只需要 x 和 y 就能唯一确定一个点。

翻转操作相当于异或，我们自然地在 $\mathbb{F}_2$ 域上讨论问题。

将网格状态用定义在 $\mathbb{F}_2$ 域上的多项式表示：

$$S(x, y) = \sum_{(u, v, -u-v) \text{ 是黑点}} x^u y^v$$

操作一次对应的多项式：

$$\begin{aligned} P(x, y) &= 1 + x + y + x^{-1} + y^{-1} + xy^{-1} + x^{-1}y \\ &= (1 + x + y)(1 + x^{-1} + y^{-1}) \end{aligned} \tag{1}$$

(2)

对任意一点 (u, v) 进行操作，相当于在原状态上加上 $x^u y^v P(x, y)$ 。例如，选择了 $(u_1, v_1), (u_2, v_2)$ 作为中心进行了翻转，操作后即 $(x^{u_1} y^{v_1} + x^{u_2} y^{v_2})P(x, y)$ 。

一个初始状态 $S(x, y)$ 能被消除当且仅当 $S(x, y)$ 能被 $P(x, y)$ 整除。

对 $1 + x + y$ 和 $1 + x^{-1} + y^{-1}$ 分别做整除性检验，分别令 $y = 1 + x$ 和 $y = \frac{x}{x+1}$ 代入 $S(x, y)$ ，得到一元多项式 $F_1(x)$ 和 $F_2(x)$ ，它们必须都恒等于 0。

选取一个足够大的有限域 $\mathbb{F}_{2^{64}}$ ，一个非零多项式在有限域上随机取值，结果为 0 的概率极低（Schwartz-Zippel Lemma）。任取随机数 α ，检验是否 $F_1(\alpha) = F_2(\alpha) = 0$ 即可。若不放心还可取 $2 \sim 3$ 个 α 。

在 $\mathbb{F}_{2^{64}}$ 下，加法即异或，乘法可以在模一个 64 次不可约多项式下进行，求幂需要预处理 α^{2^k} 利用倍增快速幂运算。时间复杂度 $O(n \log^2 w)$ 。

H. 不降子段划分

逆序维护删除变成添加整数，每次添加考虑离添加位置最近的左边和右边一个元素与待添加元素的大小关系维护答案即可。查询待添加位置的前驱和后继可以利用平衡树或并查集维护。

时间复杂度： $\mathcal{O}(n \log n)$ 或 $\mathcal{O}(n \alpha(n))$ ，空间复杂度： $\mathcal{O}(n)$ 。

I. 水壶检测

题目背景改编自本人寒假时的经历，不幸的灵感和不幸的水平结合催生了不幸的题目。大奋题致歉。

题目重述

给定数码管的参数，要求找若干个数字使其与初始温度一起得到的显示值的并为 k 个 8。

Sol 1

容易观察到每个位的覆盖是独立的。

同时注意到，对于一位，最多只需要 3 个数字就可以覆盖（例如选择 1, 2, 6）。最多只需要三个数就可以覆盖所有情况。

枚举左温度，对左温度和初始温度的并的各位处理右温度应该有哪些数字被选择（以覆盖漏掉的段），对数位集合进行搜索。

对一个比较粗糙的实现开了 3 倍时限。

Sol 2

k 比较小，并且若干个数字组合能得到的可能数十分有限，考虑直接对（温度，覆盖集）爆搜，注意剪枝。

J. 约会

注意到一条边至多被访问一次，否则后面访问这条边的时候就会产生两个已经访问过的节点，而每次访问的节点数量是 边数 + 1。

于是如果答案是 Yes 的话，我们直接每次暴力遍历所有经过的节点即可达到 $O(n + m)$ 的复杂度，如果有多个之前已经访问过的节点的话就可以标记答案为 No 了，（注意如果要提前退出程序之前需要处理完本次的读入）。

K. 最大化伤害

- 规定变换后的序列是 $b_1, b_2, b_3, \dots, b_n$ ，也就是 $b_i = \gcd(a_1, a_2, \dots, a_i)$ ， $b_{i+1} | b_i$
- 如果我们规定 $b_k = m$ ，那么 $m | a_i (i \leq k)$ ；
- 我们将 $\{b_1, b_2, b_3, \dots, b_n\}$ 去重后会变为 $\{c_1, c_2, \dots, c_q\}$ ，并且 $c_{i+1} | c_i$ ，那么 we 可知 c_i 其实对应着 $\{b\}$ 中的一段区间，记为 $[l_i, r_i]$
- 贪心可知，若 $c_j | a_i$ ，那么 $i \leq r_j$
- 从而我们可以得到 dp 思路，设 $cnt[i]$ 表示 a 中 i 这个数的出现频率：

$$dp[x] = \max_{x|y} (dp[y] + (cnt[x] - cnt[y]) * x)$$

L. 不要再 Ultra 了

$\mathcal{O}(n)$ 递推

设第 i 次操作后随机变量为 v_i ，并记其期望为 $E_i = \mathbb{E}[v_i]$ ，初值 $E_0 = 0$ 。

每一步有两种情况：

- 以概率 r 成功： $v_i = v_{i-1} \cdot k + y$
- 以概率 $1 - r$ 失败： $v_i = 0$

对期望直接列递推：

$$E_i = r \cdot (kE_{i-1} + y) + (1 - r) \cdot 0 = (rk)E_{i-1} + ry$$

记 $\alpha = rk$, $\beta = ry$ ，则

$$E_i = \alpha E_{i-1} + \beta$$

时间复杂度 $\mathcal{O}(n)$ ，空间复杂度 $\mathcal{O}(1)$ 。

$\mathcal{O}(\log n)$ 闭式

同样由

$$E_i = (rk)E_{i-1} + ry$$

可得等比求和形式

$$E_n = ry \sum_{j=0}^{n-1} (rk)^j$$

于是

$$E_n = \begin{cases} n \cdot ry, & rk = 1 \\ ry \cdot \frac{(rk)^n - 1}{rk - 1}, & rk \neq 1 \end{cases}$$

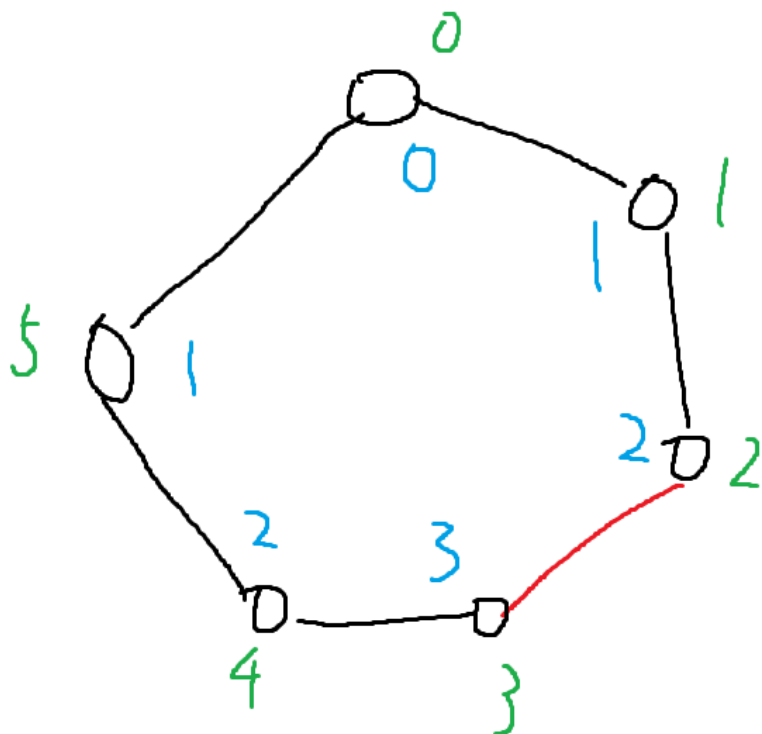
注意特判 $rk = 1$ （模意义下同样是判断 $rk \equiv 1 \pmod{M}$ ），避免分母为 0。

该做法时间复杂度 $\mathcal{O}(\log n)$ ，空间复杂度 $\mathcal{O}(1)$ 。

M. 阿基维利（交互）

在一棵树上新加了一条边之后，树就变成了基环树，即新加的边必定在给定的图的唯一简单环上。因此，可以先用 dfs 或者 bfs 等方式把这个简单环先找出来，问题就变成了在环上找出新加的边。

那么对于一个环而言，假设我们固定一个点作为起点，去求这个起点按顺时针到其他点的距离的话，一定会在新加的边那里产生“突变”，如下图，红色边是新添加的边，绿色点是以最上面的点作为起点，顺时针走到其他点的距离，蓝色点则是实际距离：



可以发现，蓝色与绿色不匹配时，说明新添加的额外边就在附近。由上面可知，找到这样的位置是可以二分，即找到第一个蓝色不等于绿色的位置，然后再在附近的节点进行查找即可。询问次数不超过 $\log n$ 次，30 次的限制绰绰有余。

N. Subarray MEX

先说结论，除了 $m = 1$ 的部分情况，其余所有情况均有解，以下为一种参考构造方案：

按 m 的值讨论以下三类：

- $m = 1$ 时，我们只能填 0、1 两种数字，题目可以转化为，构造长度为 n 的 01 数组，使得恰有 k 个子数组只包含 0。容易发现只有连续的 0 能提供有效区间，连续的 L 个 0 可提供 $\frac{L(L+1)}{2}$ 个有效区间，而不同组相同的 0 之间需要用至少 1 个 1 隔开。这可以建模成一个数学问题，我们有 $n + 1$ 个空间，消耗 L 个空间可以获得 $\frac{L(L-1)}{2}$ 个有效区间，我们需要使得总有效区间 $= k$ 。因为多余空间我们可以用 1 填充，所以如果构造出 k 个有效区间所需的最小空间 $\leq n + 1$ 则有解，否则无解。可以考虑 dp 解决，定义 dp_i 表示构造出 i 个有效区间所需的最小空间，递推时暴力枚举即可，时间复杂度 $O(n^3)$ ，状态转移时保留每个状态的前驱状态即可反推出方案。由于该 dp 表适用所有 n, k ，只需要预处理出来即可，每次对于不同的 n, k 都可以用该表 $O(n)$ 推出方案，于是总复杂度 $O(500^3 + \sum n)$
- $m = 2$ 时，首先，如果我们构造交叉数组 $[0, 1, 0, 1, \dots]$ ，那么有效区间达到上界，也就是 $\frac{n(n-1)}{2}$ ，而如果出现两个相同的相邻，其余规律不变，如 $[0, 1, 1, 0, 1, 0, 1, \dots]$ ，不难发现有效区间只减少了一个，也就是相邻的那两个 1。于是很容易得出结论，对于每个长度为 L 的，连续的 0/1，可以使得有效区间从上界开始减少 $\frac{L(L-1)}{2}$ 个。而我们的目的是使得有效区间减少恰好 $\frac{n(n-1)}{2} - k$ 个。又是类似的问题，与刚刚有细微差别，因为我们这次不需要其余数字作为分割。也就是说，我们有 n 个空间，一次性消耗 L 个空间可以减少 $\frac{L(L-1)}{2}$ 个有效区间。很明显，如果只依靠这个是无法做到满足所有 k 值的。如果我们需要减少的有效区间接近 $\frac{n(n-1)}{2}$ 个，很多情况下是无解的。然而，与 $m = 1$ 的情况不同，我们可以使用数字 2。明显，如果某个子数组包含 2，那么它的 MEX 一定不为 2。既然需要减少的有效区间很大的情况下无法做到，那我们就将其减少。如果我们令数组中第一个数为 2，那么实际上问题变成：构造一个长度为 $n - 1$ 的数组，使得有效区间数量为 k ，那么我们需要减少的有效区间数量就变为 $\frac{(n-1)(n-2)}{2} - k$ ，明显变少了。于是我们可以不断地填入连续的 2 从而让 n 不断减小，但需保证 $\frac{n(n-1)}{2} \geq k$ 。

现在，假设我们将 n 减小到不能再减小，也就是 $\frac{n(n-1)}{2} \geq k$ 且 $\frac{(n-1)(n-2)}{2} < k$ ，那么 $\frac{n(n-1)}{2} - k$ 最大是多少呢？很明显，就是 $\frac{n(n-1)}{2} - \frac{(n-1)(n-2)}{2} - 1 = n - 2$ 。

查表：

L	$\frac{L(L-1)}{2}$
1	0
2	1
3	3

容易发现，用 $n (\geq 2)$ 减少最多 $n - 2$ 个有效区间，总存在解，并且贪心即可达到。

实现可以二分搜索，时间复杂度 $O(n \log n)$ ，也可以用双指针，时间复杂度 $O(n)$ 。

- $m \geq 3$ 时，可以完全不使用 m 即可构造出来。

优先考虑 $m = 3$ 的情况。参考 $n = 8$ 的例子，我们先构造 $a = [0, 1, 2, 0, 1, 2, 0, 1]$ 。这样的循环结构使得答案达到上限，而不难发现每个索引作为左端点的有效区间数量分别为 $b = [6, 5, 4, 3, 2, 1, 0, 0]$ 。此时，如果我们修改最后一个数字，将数组变为 $a' = [0, 1, 2, 0, 1, 2, 0, 0]$ ，只有倒数第 3 个索引受到影响，每个索引作为左端点的有效区间数量分别为 $b' = [6, 5, 4, 3, 2, 0, 0, 0]$ ，总答案恰好减去 1。这是由于循环构造的特性，修改尾端元素相当于延迟最后一个有效循环结的出现，延迟距离恰好为 1，这也就使得有效区间数量减去 1。于是我们盯着最末尾的有效循环结，继续手玩即可发现如下规律：

构造数组	以各索引作为左端点的有效区间数
$[0, 1, 2, 0, 1, 2, 0, 1]$	$[6, 5, 4, 3, 2, 1, 0, 0], \sum = 21$
$[0, 1, 2, 0, 1, 2, 0, 0]$	$[6, 5, 4, 3, 2, 0, 0, 0], \sum = 20$
$[0, 1, 2, 0, 1, 2, 2, 0]$	$[6, 5, 4, 3, 1, 0, 0, 0], \sum = 19$
$[0, 1, 2, 0, 1, 2, 2, 2]$	$[6, 5, 4, 3, 0, 0, 0, 0], \sum = 18$
$[0, 1, 2, 0, 1, 1, 2, 2]$	$[6, 5, 4, 2, 0, 0, 0, 0], \sum = 17$
$[0, 1, 2, 0, 1, 1, 1, 2]$	$[6, 5, 4, 1, 0, 0, 0, 0], \sum = 16$
$[0, 1, 2, 0, 1, 1, 1, 1]$	$[6, 5, 4, 0, 0, 0, 0, 0], \sum = 15$
...	...

$m > 3$ 的情况与此类似，可以推广，代码实现复杂度 $O(n)$ ，详细可参考代码。

O. 最长好数区间

考虑如果一个整数所有后缀都是质数的情况可能不多（因为也要保证这个数是质数），因此先打表观察 10^9 范围内有多少数不是好的，这一步可以利用 Trie 树进行搜索。实际上「不好的」整数很少，约 18000 多个（下文记为 w 个），除此之外的整数都是好的，因此最大范围内只有 $O(w)$ 个好数区间，从左到右枚举或二分寻找在 $[L, R]$ 范围内的好数区间，记录最大长度更新答案即可。

注意到大于 10 的「不好的」整数，其末位一定是 3 或 7，可以利用此性质进行剪枝。

P. 旅行计划

题意描述：有 n 个地点，有 $a_i * x$ 货币，每天花费 b_i ，第一天免费，最后 n 个地点的天数总和为 c ，求正整数 x 的个数。

题解：对于一个地点，天数为 $(a_i * x / b_i) + 1$ 结果下取整，总和为

$$\sum_{i=1}^n \left\lfloor \frac{a_i x}{b_i} \right\rfloor$$

二分使得上式小于等于 c 的最大 x ，再对于 $c - 1$ 二分一次，答案就是两者的差值。

时间复杂度为 $O(n \log X)$ X 为 x 的最大值。

Q. 许愿池

一个典型错误是利用 $\{p_n\}$ 的通项公式计算，由于 $a - 1$ 在模 c 意义下的逆元不一定存在，因此直接计算可能有误。

考虑构造如下矩阵

$$A = \begin{bmatrix} a & -1 \\ 0 & 1 \end{bmatrix}, b_n = \begin{bmatrix} p_n \\ a \end{bmatrix}$$

则 $b_{n+1} = Ab_n$ ，也就是说 $b_{n+1} = A^n b_1$ 。利用矩阵快速幂计算即可。

时间复杂度： $\mathcal{O}(\log n)$ ，空间复杂度： $\mathcal{O}(1)$ 。